

PROJECT PROPOSAL

AWS IoT Monitoring and Control Dashboard

Cloud-connected environmental monitoring and remote device control using YOLO UNO, FastAPI, PostgreSQL, React, and AWS.

English edition

Prepared for First Cloud Journey / AWS Study Group

Proposal date: 15/06/2026

1. Executive Summary

The **AWS IoT Monitoring and Control Dashboard** is a cloud-connected IoT platform designed to monitor environmental data and remotely control devices in a room.

The system uses a **YOLO UNO ESP32-S3** device to collect temperature, humidity, and light intensity data. The hardware sends telemetry through HTTP REST APIs to a **FastAPI backend hosted on Amazon EC2**. The backend stores telemetry and device commands in **Amazon RDS for PostgreSQL**.

A **React + Vite dashboard** displays the latest telemetry, historical data, device states, and command results. Users can control a fan, light, and curtain through the dashboard. The device periodically polls the backend for pending commands, executes them, and sends an acknowledgement so that the command state changes from **Pending** to **Executed**.

Amazon CloudWatch collects backend logs and infrastructure metrics. CloudWatch Alarms are configured to monitor EC2 and RDS operating conditions.

2. Problem Statement

2.1 Current Problem

Small rooms, laboratories, and learning environments often use sensors and electrical devices independently. This creates several limitations:

- Environmental data is not stored centrally.
- Users cannot review historical temperature, humidity, or light data.
- Devices cannot be controlled remotely from one dashboard.
- A control request does not always prove that the physical device executed the command.
- System logs and infrastructure metrics are difficult to monitor without a centralized platform.

2.2 Proposed Solution

The project provides a cloud-based monitoring and control platform with a complete two-way data flow:

The platform supports:

- Telemetry ingestion from physical hardware.
- Latest and historical sensor data.
- Remote fan, light, and curtain control.
- Command tracking with **Pending** and **Executed** states.
- Device acknowledgement after command execution.
- Rule-based analysis and control recommendations.
- Monitoring with CloudWatch Logs, Metrics, and Alarms.

2.3 Benefits

- Centralized monitoring for sensor and actuator data.
- Two-way communication between the dashboard and physical hardware.
- Persistent telemetry and command history in PostgreSQL.
- Better troubleshooting through logs, metrics, and command states.
- A modular design that can be expanded to additional rooms and devices.

3. Project Objectives and Scope

3.1 Objectives

The project aims to:

1. Build a physical IoT device using YOLO UNO ESP32-S3.
2. Collect temperature, humidity, and light sensor data.
3. Send telemetry to a FastAPI REST API on Amazon EC2.
4. Store telemetry and commands in Amazon RDS for PostgreSQL.
5. Display latest and historical data on a React dashboard.
6. Remotely control a fan, light, and curtain.
7. Implement the command lifecycle **Pending** → **Executed**.
8. Send command acknowledgements from the device.
9. Monitor EC2, RDS, and backend logs with Amazon CloudWatch.
10. Produce a bilingual workshop and project report.

3.2 Project Scope

The current implementation focuses on one sample device:

```
DEVICE_ID = room_01
```

The device includes:

- DHT20 temperature and humidity sensor.
- Analog light sensor.
- Fan module.
- Light or relay module.
- Servo motor for curtain control.

Supported commands:

FAN_ON
FAN_OFF
LIGHT_ON
LIGHT_OFF
CURTAIN_OPEN
CURTAIN_CLOSE

3.3 Out of Scope

The current version does not use:

- AWS IoT Core.
- AWS Lambda.
- Amazon API Gateway.
- Amazon S3.
- Amazon SNS.
- Amazon ECS or ECR.
- Amazon Cognito.
- Amazon CloudFront.
- Amazon DynamoDB.

These services may be considered in future versions if the project requires a different deployment model.

4. Solution Architecture

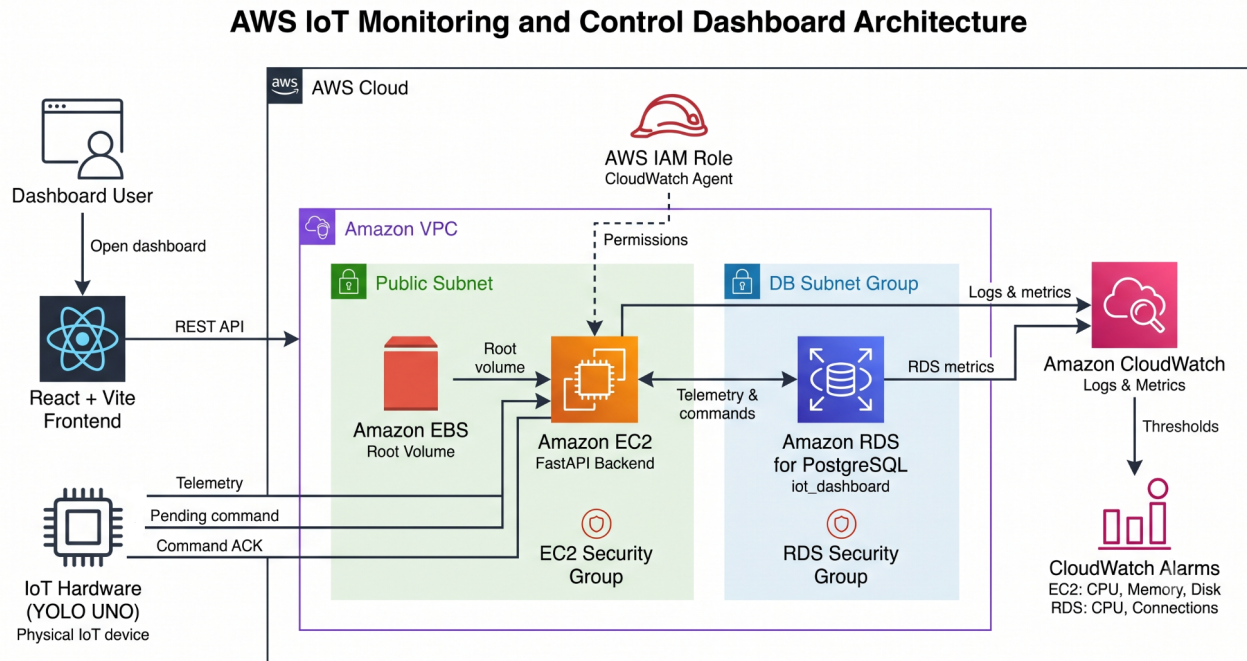


Figure 1. AWS IoT Monitoring and Control Dashboard architecture.

4.1 Architecture Components

Component	Technology	Purpose
Frontend	React, Vite, TypeScript, Tailwind CSS	Displays telemetry, history, analysis, and device controls
Backend	Python, FastAPI, Uvicorn, SQLAlchemy, Pydantic	Provides REST APIs and processes telemetry, commands, and ACKs
Database	Amazon RDS for PostgreSQL	Stores telemetry records and command states
Compute	Amazon EC2 with Amazon EBS	Runs the FastAPI backend as a systemd service
IoT Hardware	YOLO UNO ESP32-S3, PlatformIO, Arduino	Reads sensors, controls actuators, polls commands, and sends ACKs
Networking	Amazon VPC, subnets, Security Groups	Controls network access between users, EC2, and RDS
Identity	AWS IAM Role	Grants the EC2 instance permission to publish CloudWatch data
Monitoring	Amazon CloudWatch and CloudWatch Alarms	Collects logs and metrics and evaluates configured thresholds

4.2 Data Flow

Telemetry Flow

1. YOLO UNO reads temperature, humidity, light intensity, and actuator states.
2. The device sends telemetry to:

```
POST /api/telemetry
```

1. FastAPI validates the payload.
2. The backend stores the data in Amazon RDS for PostgreSQL.
3. The frontend retrieves the latest or historical data from the backend.

Command Flow

1. A user presses a control button on the dashboard.
2. The frontend sends a command to:

```
POST /api/devices/{device_id}/commands
```

1. The backend stores the command with the **Pending** state.
2. YOLO UNO polls:

```
GET /api/devices/{device_id}/commands/latest
```

1. The device executes the command.
2. The device sends an acknowledgement to:

```
POST /api/devices/{device_id}/commands/{command_id}/ack
```

1. The backend updates the command state to **Executed**.

4.3 Service Selection

AWS Service	Selection Reason
Amazon EC2	Provides full control of the FastAPI runtime, Python environment, systemd service, and log files
Amazon EBS	Provides persistent root storage for the EC2 instance
Amazon RDS for PostgreSQL	Managed relational database suitable for telemetry and command records
Amazon VPC	Provides network isolation and Security Group control
AWS IAM Role	Avoids hard-coded AWS access keys on EC2
Amazon CloudWatch	Centralizes logs, metrics, and alarms for operations and troubleshooting

4.4 Security Design

- SSH access is restricted to the administrator IP.
 - The backend port is opened only as required for the project demo.
 - RDS accepts PostgreSQL traffic only from the EC2 Security Group.
 - AWS access keys are not hard-coded in the application.
 - EC2 uses an IAM Role for CloudWatch Agent permissions.
 - `.env`, `.pem`, private keys, and `hardware/include/secrets.h` are excluded from Git.
 - Public documentation uses placeholders instead of real credentials.
-

5. Technical Implementation Plan

Phase 1 — Cloud Foundation

- Design the AWS architecture.
- Configure VPC, subnet, Security Groups, and IAM Role.
- Launch the EC2 instance.
- Create Amazon RDS for PostgreSQL.
- Verify EC2-to-RDS connectivity.

Phase 2 — Backend and Database

- Initialize the FastAPI backend.
- Define Pydantic schemas and SQLAlchemy models.
- Implement telemetry ingestion.
- Implement latest and historical data APIs.
- Implement command creation, polling, and acknowledgement.
- Run the backend as the `aws-iot-backend` systemd service.

Phase 3 — Hardware

- Configure the YOLO UNO board in PlatformIO.
- Connect DHT20, analog light sensor, fan, light, and servo.
- Connect the board to Wi-Fi.
- Send telemetry to the EC2 backend.
- Poll pending commands.
- Execute device commands.
- Send ACKs and prevent duplicate command execution.

Phase 4 — Frontend and Integration

- Build the React + Vite dashboard.
- Display latest and historical telemetry.
- Add fan, light, and curtain controls.
- Add manual and rule-based automatic or recommendation modes.

- Integrate the frontend with the EC2 backend.
- Debug the complete system flow.

Phase 5 — Monitoring and Documentation

- Install and configure CloudWatch Agent.
- Collect backend logs, EC2 memory, and disk metrics.
- Monitor EC2 CPU and RDS metrics.
- Create CloudWatch Alarms.
- Perform end-to-end testing.
- Record the demonstration video.
- Complete the bilingual report and workshop.

6. Timeline and Milestones

The project is planned over **10 weeks**.

Week	Milestone	Expected Output
Week 1	Requirements analysis and project planning	Problem statement, scope, team roles, initial architecture
Week 2	AWS architecture and network foundation	VPC, subnets, Security Groups, IAM plan
Week 3	EC2 and RDS deployment	Running EC2 instance and PostgreSQL database
Week 4	Backend and database foundation	FastAPI structure, schemas, database connection
Week 5	Telemetry and command APIs	Telemetry, latest, history, command, and ACK endpoints
Week 6	YOLO UNO hardware integration	Sensor readings, actuator control, Wi-Fi, REST communication
Week 7	Frontend dashboard development	Telemetry cards, charts, control panel, analysis section
Week 8	End-to-end integration and debugging	Complete telemetry and command flow with physical hardware
Week 9	CloudWatch monitoring and system validation	Logs, metrics, alarms, failure tests, security review
Week 10	Documentation, demo, and final handover	Bilingual report, workshop website, demo video, final repository

7. Budget Estimation

The project uses small AWS resources suitable for learning and demonstration.

Resource	Cost Factor	Optimization Approach
Amazon EC2	Instance running time	Use a small instance and stop it when it is not required
Amazon EBS	Provisioned storage	Allocate only the required root volume size
Amazon RDS for PostgreSQL	Instance class, storage, and running time	Use a small database instance for the workshop environment
Amazon CloudWatch	Log ingestion, retention, and custom metrics	Limit log retention and avoid unnecessary high-frequency metrics
Data Transfer	Requests between users, hardware, and EC2	Use reasonable telemetry and polling intervals

The exact cost depends on region, resource size, and usage duration. Resources must be reviewed and cleaned up after the workshop to avoid unexpected charges.

8. Risk Assessment

Risk	Impact	Mitigation
YOLO UNO loses Wi-Fi	Telemetry and commands stop temporarily	Reconnect to Wi-Fi and retry HTTP requests
EC2 public IP changes	Frontend and hardware cannot reach the backend	Update configuration or attach an Elastic IP in a future improvement
Backend process stops	APIs become unavailable	Run Uvicorn with <code>systemd</code> and automatic restart
RDS connection failure	Telemetry and commands cannot be stored	Verify endpoint, credentials, Security Groups, and <code>DATABASE_URL</code>
Duplicate command execution	Actuators may repeat an action	Store the latest command ID and execute each command only once
ACK request fails	Command remains <code>Pending</code>	Retry the ACK without repeating the actuator action
Credentials are exposed	Security incident	Ignore <code>.env</code> , <code>.pem</code> , private keys, and <code>secrets.h</code> in Git

Risk	Impact	Mitigation
Unexpected AWS cost	Project budget increases	Stop or delete resources after testing and review CloudWatch usage
Incorrect sensor values	Dashboard recommendations become inaccurate	Validate readings and calibrate sensor thresholds
Integration mismatch	Frontend, backend, and hardware use different payloads or endpoints	Maintain one API contract and test each endpoint end to end

9. Expected Results

The project is expected to deliver:

- A working YOLO UNO physical IoT device.
- Periodic temperature, humidity, and light telemetry.
- A FastAPI backend running on Amazon EC2.
- Persistent telemetry and command data in Amazon RDS for PostgreSQL.
- A React dashboard with latest data, history, controls, and recommendations.
- Remote fan, light, and curtain control.
- Command state tracking from **Pending** to **Executed**.
- Device acknowledgement for completed commands.
- CloudWatch logs, metrics, and alarms.
- A bilingual FCAJ workshop and final report.
- A demonstration video showing the dashboard, database, CloudWatch, and physical hardware.

Success Criteria

The project is considered successful when:

1. YOLO UNO can send valid telemetry to the backend.
 2. Telemetry is stored and retrieved from PostgreSQL.
 3. The dashboard displays current and historical data.
 4. The dashboard creates commands for **room_01**.
 5. The hardware receives and executes supported commands.
 6. ACK updates commands to **Executed**.
 7. CloudWatch receives the configured logs and metrics.
 8. Another reader can follow the workshop and reproduce the main deployment steps.
-

10. Team Responsibilities

Member	Role	Primary Responsibilities
Pham Le Minh Khoi	AWS and Hardware Lead	AWS architecture, VPC, Security Groups, IAM Role, EC2, RDS, CloudWatch, DevOps, YOLO UNO firmware, sensors, actuators, telemetry, command polling, and ACK
Ngo Minh Thuan	Backend Developer	FastAPI backend, API endpoints, Pydantic schemas, SQLAlchemy models, PostgreSQL integration, telemetry processing, command lifecycle, and ACK processing
Thuong Dinh Hung	Frontend and Integration Developer	React + Vite dashboard, user interface, telemetry visualization, device controls, overall system integration, debugging, and demo video recording
Le Bao Khanh	Documentation and QA	Proposal, blog posts, weekly worklog, event reports, workshop documentation, bilingual content review, navigation, screenshots, and quality assurance

11. Deliverables

The final project deliverables include:

- Source code repository.
- FastAPI backend.
- React + Vite frontend.
- YOLO UNO PlatformIO firmware.
- AWS architecture diagram.
- Amazon RDS database schema.
- CloudWatch logs, metrics, and alarms.
- API documentation.
- Bilingual proposal and workshop.
- Worklog, blog posts, and event reports.
- End-to-end demonstration video.
- Clean-up and project handover instructions.